

```

1 # Part 1 Step by Step
2
3 # 1. Loading NYC taxi trip data
4 # 2. Loading tourist locations
5 # 3. Converting both into spatial coordinates
6 # 4. Classifying trips as: tourist-related vs. non-tourist
7 # 5. Computing: trip distance and speed
8 # 6. Comparing average speeds between groups
9
10 import json
11 from pathlib import Path
12 import duckdb
13
14 BASE_DIR = Path(__file__).resolve().parent
15 DATA_DIR = BASE_DIR / "data"
16 DB_PATH = BASE_DIR / "taxi_analysis.duckdb"
17
18 # Constants
19 FEET_PER_MILE = 5280.0
20 TOURIST_THRESHOLD_FEET = 0.1 * FEET_PER_MILE
21 MAX_SPEED_MPH = 100
22
23 # Make DuckDB's spatial functions available
24 def load_spatial(conn: duckdb.DuckDBPyConnection) -> None:
25     conn.execute("INSTALL spatial;")
26     conn.execute("LOAD spatial;")
27
28 # Connect to the DuckDB database and loads the spatial extension.
29 # Create the taxi_trips table from the CSV and tourist_spots from GeoJson
30 def main() -> None:
31     print("Part 1: Tourist vs. non-tourist taxi trip speeds")
32
33     # Opens/creates database and loads spatial features to it
34     conn = duckdb.connect(str(DB_PATH))
35     load_spatial(conn)
36
37     # Load taxi trip data into a table in DB
38     conn.execute(
39         f"""
40         CREATE OR REPLACE TABLE taxi_trips AS
41         SELECT *
42         FROM read_csv_auto('{DATA_DIR / "nyc_taxi_data.csv"}', header=true)
43         """
44     )
45
46     # Load tourist spots into python dictionary
47     with open(DATA_DIR / "tourist_locations-1.geojson") as f:
48         tourist_locations = json.load(f)
49
50     # Create new table with tourist spots in DB
51     conn.execute("DROP TABLE IF EXISTS tourist_spots")
52     conn.execute(
53         """
54         CREATE TABLE tourist_spots (
55             name VARCHAR,
56             geom GEOMETRY
57         )
58         """
59     )
60
61     # Loop through each tourist location and extract coordinates
62     for feature in tourist_locations["features"]:
63         longitude, latitude = feature["geometry"]["coordinates"]

```

```

64      # Insert tourist spot name, geometry point, transformed point
65      conn.execute(
66          """
67          INSERT INTO tourist_spots
68          VALUES (
69              ?,
70              ST_Transform(ST_Point(?, ?), 'EPSG:4326', 'EPSG:2263')
71          )
72          """,
73          [feature["properties"].get("Tourist_Spot", ""), longitude, latitude],
74      )
75
76      conn.execute(
77          """
78          CREATE OR REPLACE TABLE part1_trip_speeds AS
79          WITH trip_geometries AS (
80              SELECT
81                  id,
82                  trip_duration,
83                  ST_Transform(
84                      ST_Point(pickup_longitude, pickup_latitude),
85                      'EPSG:4326',
86                      'EPSG:2263'
87                  ) AS pickup_geom,
88                  ST_Transform(
89                      ST_Point(dropoff_longitude, dropoff_latitude),
90                      'EPSG:4326',
91                      'EPSG:2263'
92                  ) AS dropoff_geom
93          FROM taxi_trips
94          WHERE
95              pickup_longitude IS NOT NULL
96              AND pickup_latitude IS NOT NULL
97              AND dropoff_longitude IS NOT NULL
98              AND dropoff_latitude IS NOT NULL
99              AND trip_duration > 0
100      ),
101      classified AS (
102          SELECT
103              *,
104              CASE
105                  WHEN EXISTS (
106                      SELECT 1
107                      FROM tourist_spots
108                      WHERE ST_Distance(pickup_geom, geom) <= ?
109                  )
110                  OR EXISTS (
111                      SELECT 1
112                      FROM tourist_spots
113                      WHERE ST_Distance(dropoff_geom, geom) <= ?
114                  )
115                  THEN 'tourist'
116                  ELSE 'non-tourist'
117              END AS location
118          FROM trip_geometries
119      )
120      SELECT
121          id,
122          location,
123          ST_Distance(pickup_geom, dropoff_geom) / ? AS distance_miles,
124          (ST_Distance(pickup_geom, dropoff_geom) / ?) / (trip_duration / 3600.0) AS speed_mph
125      FROM classified
126      """,
127      [TOURIST_THRESHOLD_FEET, TOURIST_THRESHOLD_FEET, FEET_PER_MILE, FEET_PER_MILE],
128  )

```

```
129
130     result = conn.execute(
131         """
132         SELECT
133             location,
134             AVG(speed_mph) AS average_mph
135         FROM part1_trip_speeds
136         WHERE speed_mph > 0 AND speed_mph <= ?
137         GROUP BY location
138         ORDER BY CASE WHEN location = 'tourist' THEN 0 ELSE 1 END
139         """
140         ,
141         [MAX_SPEED_MPH],
142     ).fetchdf()
143
144     output_path = BASE_DIR / "part1.csv"
145     result.to_csv(output_path, index=False)
146     print(result)
147     print(f"Wrote {output_path}")
148
149     conn.close()
150
151 if __name__ == "__main__":
152     main()
```